



NOMBRE DEL ESTUDIANTE:

Francisco Xavier Gil Ginez

DOCENTE:

José Miguel Carrera Pacheco

MATERIA:

Desarrollo Web Profesional

CUATRIMESTRE ENERO-ABRIL

2026

1. Conceptos Fundamentales

¿Qué es Autenticación y en qué se diferencia de Autorización?

Para un sistema de control de acceso, esta diferencia es crucial.

Autenticación: Es el proceso de verificar "¿Quién eres?". El repartidor presenta sus credenciales (ej: un código QR único y una contraseña de un solo uso) para demostrar su identidad. Si las credenciales son válidas, el sistema confirma que es quien dice ser.

Autorización: Es el proceso de verificar "¿Qué se te permite hacer?". Una vez autenticado, el sistema consulta: ¿Este repartidor, en este momento, tiene permiso para abrir la puerta principal? ¿Puede entrar a la calle privada 123? ¿O solo tiene acceso al área de recepción de paquetería?

En resumen, primero identificas quién llama a la puerta (autenticación) y luego decides si le abres y qué habitaciones del "castillo" puede visitar (autorización).

¿Qué son JWT, Sesiones y OAuth?

Sesiones (Session-based Authentication): Es el método "tradicional" con estado.

- El repartidor hace login. El servidor valida sus credenciales.
- El servidor crea una "sesión" en su memoria o base de datos y le asigna un ID de sesión único.
- El servidor envía este ID al dispositivo del repartidor, generalmente en una cookie.
- En cada solicitud posterior (ej: al escanear otro QR), el dispositivo envía la cookie. El servidor busca el ID de sesión en su almacenamiento para recordar quién es.

Analogía: Como un hotel que te da una llave (ID de sesión) y apunta en un gran libro (servidor) que la llave 201 es para la habitación de Juan Pérez.

JWT (JSON Web Tokens): Es un método moderno "sin estado" (stateless).

- El repartidor hace login. El servidor valida sus credenciales.

- El servidor genera un token (JWT) que contiene información del usuario (ej: su ID, rol: "repartidor") y lo firma digitalmente.
- El servidor envía este JWT al dispositivo. El dispositivo lo guarda.
- En cada solicitud posterior, el dispositivo envía el JWT. El servidor solo tiene que verificar la firma del token; no necesita buscar nada en una base de datos. Si la firma es válida y el token no ha expirado, la solicitud es de fiar.

Analogía: Como un boleto de un parque de atracciones. El parque te lo vende (lo firma) y pone en él que tienes acceso a todas las atracciones. En cada atracción, solo miran tu boleto para ver si es válido y qué acceso tienes; no necesitan llamar a la taquilla central para preguntar.

OAuth 2.0: Es un protocolo de delegación de autorización, no de autenticación directa. Se usa para que una aplicación pueda acceder a recursos en nombre de un usuario sin conocer su contraseña. El ejemplo clásico es el "Inicia sesión con Google" o "Inicia sesión con Facebook". Para este caso, es menos relevante, a menos que se desee que los repartidores se autenticuen con su cuenta de Google.

¿Cuándo conviene usar tokens y cuándo sesiones?

Característica	Sesiones (Stateful)	Tokens / JWT (Stateless)
Escalabilidad	Baja. El servidor debe almacenar y sincronizar el estado de cada usuario, lo que complica la escalabilidad horizontal.	Alta. Cualquier servidor puede procesar cualquier petición sin necesidad de memoria compartida, ideal para microservicios.
Arquitectura	Monolítica. Aplicaciones web tradicionales.	Moderna. APIs REST, Single Page Applications (SPA), Apps Móviles.
Control Inmediato	Alto. Se puede "matar" una sesión al instante (ej: al cerrar sesión o si un admin bloquea al usuario).	Bajo. Un token vive hasta que expira. Para revocarlo inmediatamente, se necesitan listas negras en servidor, lo cual rompe su naturaleza stateless.
Uso Típico	Banca online, paneles de administración donde el control estricto es vital.	APIs públicas, aplicaciones que necesitan escalar rápidamente (redes sociales, e-commerce).

Conclusión para el proyecto: Dado que habrá múltiples puntos de acceso (lectores QR) que actuarán como clientes, y potencialmente una app para repartidores, la arquitectura más adecuada es basada en **Tokens (JWT)**. Es escalable y se adapta perfectamente a una API REST que será consumida por diferentes tipos de clientes.

2. Diseño del Sistema: Control de Acceso con QR para Repartidores

Análisis de Riesgos de un Mal Diseño

Un diseño pobre podría tener consecuencias graves:

- **Suplantación de Identidad:** Si un QR es fácilmente copiable y no está vinculado a la identidad del repartidor, cualquiera que lo posea podría entrar.
- **Elevación de Privilegios:** Un repartidor podría, por error o malicia, acceder a zonas restringidas (ej: áreas comunes de residentes, otros departamentos) si el sistema no valida sus permisos correctamente.
- **Ataques de Replay:** Si un token JWT o un código QR no tiene una validez muy corta, un atacante podría interceptarlo y usarlo después para acceder.
- **Falta de Auditoría:** Si no se registra quién, cuándo y dónde accedió, es imposible investigar un incidente de seguridad.
- **Filtraciones Masivas:** Si las claves privadas con las que se firman los JWT se ven comprometidas, un atacante podría generar sus propios tokens "válidos" y hacerse pasar por cualquiera.

Definición de Usuarios, Roles y Permisos (RBAC)

Se adoptará el modelo de **Control de Acceso Basado en Roles (RBAC)**.

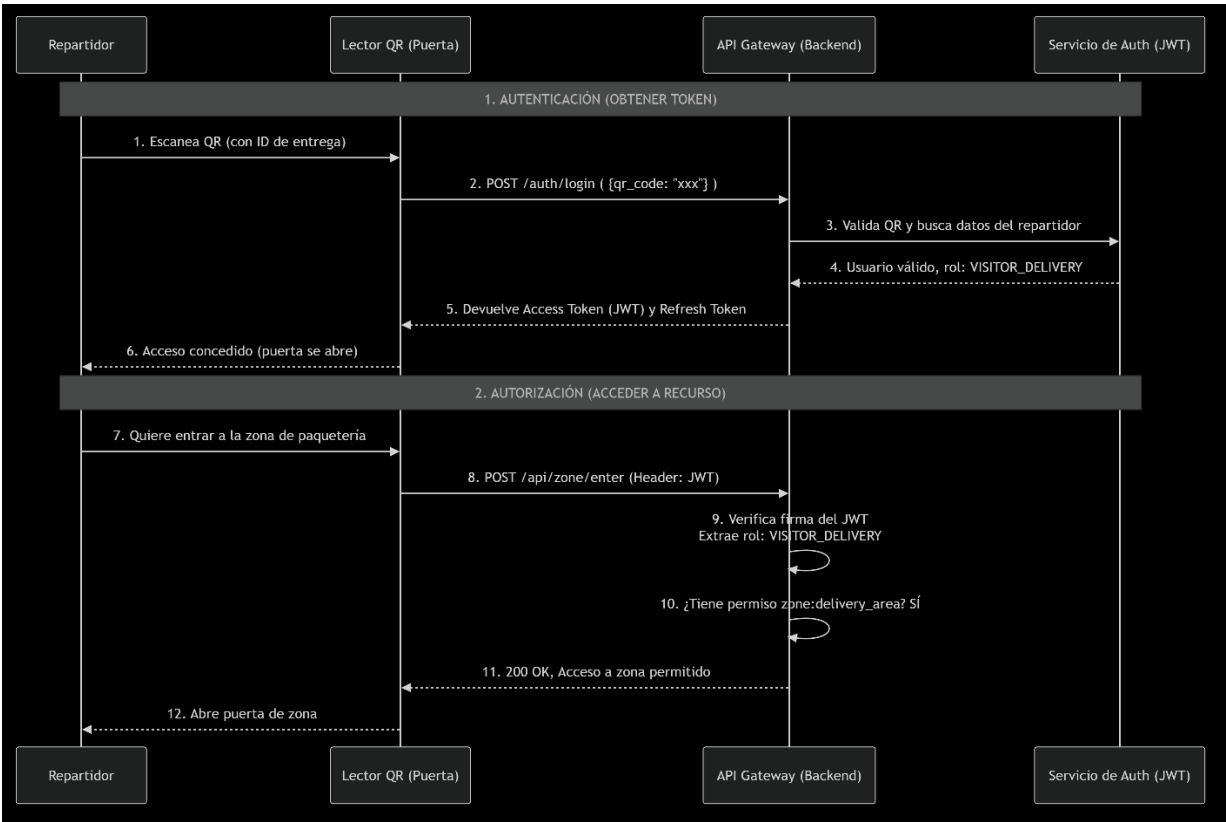
Tipos de Usuarios: Personas o sistemas que interactúan:

- **Repartidor (Humano):** Conductor de paquetería, comida, etc.
- **Administrador del Fraccionamiento (Humano):** Encargado de gestionar accesos, altas, bajas y ver logs.
- **Sistema de Cómputo (Máquina):** El lector QR en la puerta, la app del repartidor.

Roles y Permisos:

Rol	Descripción	Permisos (Ejemplos)
VISITOR_DELIVERY	Repartidor con un permiso de entrega específico.	gate:enter (solo una vez, puerta principal), zone:delivery_area (acceso a zona de paquetería)
RESIDENT_GATE (futuro)	Residente que autoriza entradas.	visitor:create_temp_qr (generar QR de un solo uso)
ADMIN	Administrador del sistema.	users:create, users:block, logs:read, roles:assign, gate:open_manual

Flujo Completo de Autenticación y Autorización



Explicación del Flujo:

1. Emisión del Código QR (Back-office): Un administrador genera un código QR para una entrega específica. Este QR no es un simple ID, sino que puede contener un identificador único de la entrega o ser un "ticket" de corta duración. Idealmente, debería estar ligado a un móvil o una identidad, para evitar que sea transferible.

2. Autenticación (Login): El repartidor escanea el QR en el lector de la puerta. El lector envía el código a la API (/auth/login). El servidor valida el código y, si es correcto, genera un par de tokens:

- Access Token (JWT): De corta duración (ej: 15-30 minutos), tiempo suficiente para completar la entrega. Contiene el user_id, el rol (VISITOR_DELIVERY) y permisos específicos (ej: gate:enter).
- Refresh Token: De larga duración (ej: 24 horas), pero solo utilizable para obtener un nuevo Access Token si la entrega se alarga inesperadamente. Se almacena de forma segura en el servidor (con estado) para poder ser revocado si es necesario.

3. Autorización (Acceso a Recursos): Cuando el repartidor quiere pasar por una segunda puerta (ej: zona de paquetería), su lector envía el Access Token en el header Authorization: Bearer <JWT>. El servidor de la API valida la firma del token. Como es un JWT, el servidor confía en la información que contiene. Verifica si el rol VISITOR_DELIVERY tiene el permiso zone:delivery_area. Si es así, concede el acceso. Este proceso es rápido y no requiere consultar una base de datos central.

3. Justificación Técnica y Mitigación de Riesgos

¿Por qué este enfoque (JWT + RBAC)?

Escalabilidad y Rendimiento: Los lectores QR (clientes) son muchos y las peticiones constantes. La naturaleza stateless de los JWT permite que cualquier instancia de la API procese la solicitud sin fricción, ideal para escalar horizontalmente cuando el fraccionamiento crezca.

Seguridad por Diseño:

- **Contra Suplantación:** Se combina el QR (algo que tienes) con la necesidad de que el repartidor, al escanearlo, pueda estar también autenticándose con un PIN o biometría en su propia app (algo que sabes/eres). Esto es Autenticación de Dos Factores (2FA/MFA). El QR sin la identidad del repartidor es inservible.
- **Contra Ataques de Replay:** Los Access Tokens tienen una vida útil muy corta (TTL bajo). Si alguien intercepta un JWT, la ventana de oportunidad para reutilizarlo es mínima.
- **Control Granular:** El modelo RBAC permite ser extremadamente específico con los permisos. Un repartidor no podrá acceder a la zona de mantenimiento porque su rol no incluye ese permiso. Esto mitiga el riesgo de elevación de privilegios.
- **Revocación y Auditoría:** Aunque los JWT son stateless, se puede mantener una lista de Refresh Tokens revocados (en una base de datos de alta velocidad como Redis). Si un administrador necesita expulsar a un repartidor, elimina su Refresh Token. El Access Token actual expirará en minutos, bloqueando eficazmente al usuario. Además, cada acción de autorización (conceder/denegar acceso) se registra en una bitácora (logs:create) para una auditoría completa.

Riesgos Mitigados

- **Acceso No Autorizado:** Mitigado por JWT de corta duración y 2FA.
- **Escalamiento de Privilegios:** Mitigado por el diseño granular de roles y permisos (RBAC).
- **Robo de Credenciales:** Mitigado por la caducidad de tokens y la posibilidad de revocación mediante Refresh Tokens.
- **Falta de Trazabilidad:** Mitigado por el registro obligatorio de cada evento de acceso.